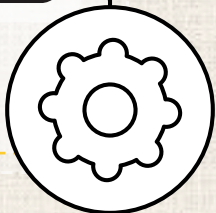


10 강

튜플, 세트, 딕셔너리

첨단공학부
정재화 교수





1. 튜플의 이해

- ✓ 튜플의 역할
- ✓ 튜플 관련 연산자

2. 세트의 이해

- ✓ 세트의 정의
- ✓ 세트 관련 함수와 메소드

3. 딕셔너리의 이해

- ✓ 딕셔너리 관련 연산자
- ✓ 딕셔너리 관련 주요 메소드



학습하기

1

튜플의 이해

튜플(tuple)

■ 데이터를 순서있게 저장하는 불변 시퀀스

- ✓ 리스트와 기능적으로 동일
- ✓ 생성 후 원소가 바뀌지 않도록 설계된 안정적 저장 구조
 - 원소의 순서 또한 고정
- ✓ 불변성으로 의도치 않은 데이터 변경을 방지
- ✓ 리스트보다 메모리 효율과 접근 속도에서 유리

구문형식

```
tuple([원소 시퀀스])  
(원소 시퀀스)
```

튜플의 역할

■ 튜플의 ()은 생략 가능

- ✓ 즉, 콤마(,)로 분리된 모든 값들은 튜플

temp, season = 27, "summer"

rad, hei = hei, rad

■ 함수는 원칙적으로 단 하나의 값만 반환

- ✓ 두 값을 ()로 묶어 한 개의 튜플로 객체화해서 전달

area, circum = circle_area_circum(rad)

■ 튜플의 생성은 괄호 ()가 아닌 콤마(,)

튜플 관련 연산자, 내장함수와 메소드

■ 할당 연산자(=) 외 연산자는 리스트와 동일

scores= (87, 61, 98, 78, 85)
score[1] = 81

■ len, min, max, sum은 리스트와 동일

■ 주요 메소드

✓ 불변성에 따라 원소가 수정되는 메소드는 미정의

메소드	의미
<code>index(x: object): int</code>	튜플 내, 원소 x의 첫 출현 위치 반환
<code>count(x: object): int</code>	튜플 내, 원소 x의 출현 횟수를 반환

튜플의 원소 추가

- 리스트는 `append()` 메소드를 통해 원소 추가
- 튜플은 불변으로 `append()` 메소드 미정의
 - ✓ + 연산자로 연결



```
scores = tuple([87, 61, 98, 78, 85])  
scores = scores + (89, )
```

튜플 슬라이싱

리스트 슬라이싱의 [start : end : step] 구문형식을 동일하게 사용

- ✓ 잘라낸 결과는 새로운 튜플로 생성
- ✓ 슬라이싱을 이용한 **부분 수정 불가**

```
scores = tuple([87, 61, 98, 78, 85])  
scores2 = scores[1 : 4]  
scores[1 : 2] = (81, 97)
```

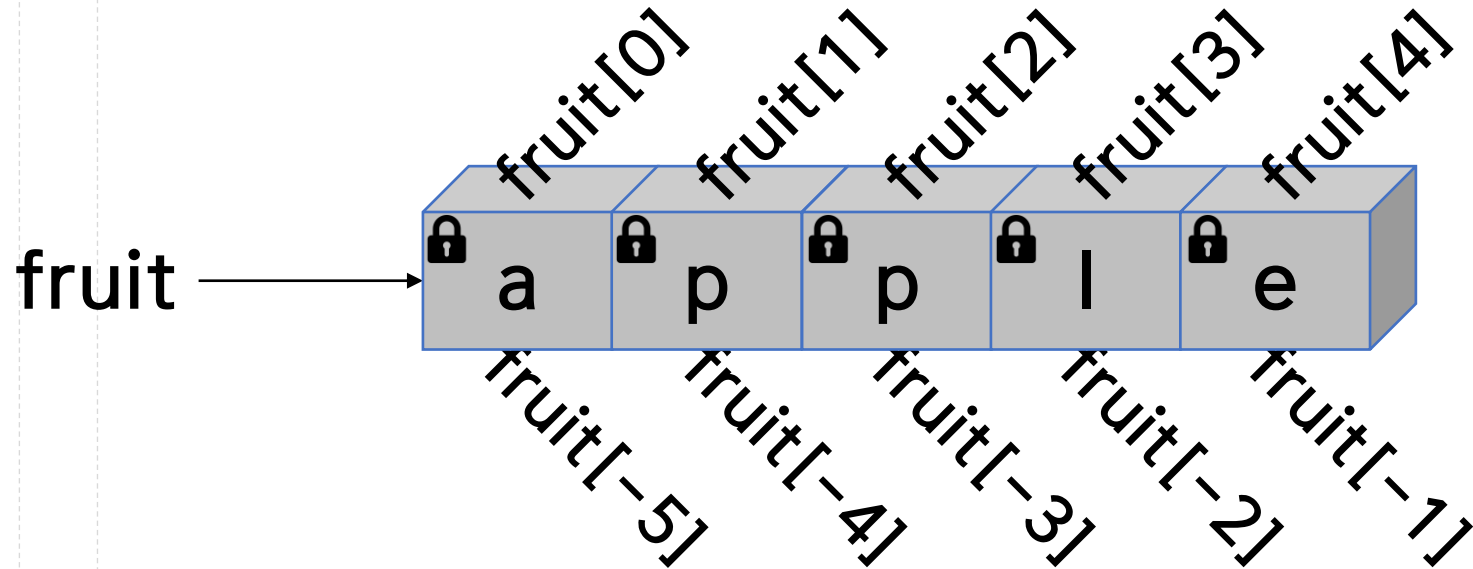


튜플과 문자열

■ 원소가 문자로만 고정된 불변 시퀀스

✓ 시퀀스의 성질을 공유

- 인덱싱, 슬라이싱, 시퀀스 내장함수, 반복 가능



```
fruit = "apple"
print("apple"[3])
```

I 문자열 메소드

메소드	의미
<code>split(str): list</code>	특정 문자를 기준으로 쪼개어 리스트로 변환
<code>join(list): str</code>	리스트 등의 문자열들을 하나로 합침
<code>find(str): int</code>	찾는 문자가 없으면 -1을 반환 (에러 X)
<code>replace(old: str, new: str): str</code>	특정 문자를 다른 문자로 바꾼 새 문자열 반환
<code>strip(): str</code>	양쪽 공백(또는 특정 문자)을 제거
<code>upper(): str</code> <code>lower(): str</code>	대문자/소문자로 변환
<code>isdigit(): bool</code>	숫자로만 구성되어 있는지 확인 (True/False)
<code>isalpha(): bool</code>	알파벳으로만 구성되어 있는지 확인
<code>isalnum(): bool</code>	알파벳과 숫자로만 구성되어 있는지 확인

문자열과 리스트

msg → I do love python

msg_list → I do love python

msg_list2 → I do love python

```
msg = "I do love python"  
msg_list = msg.split(" ")  
msg_list2 = "I do love python".split(" ")
```

리스트와 문자열

scores →

1	5	14	26	31
---	---	----	----	----

scores_csv →

1	,	5	,	14	,	26	,	31
---	---	---	---	----	---	----	---	----

```
scores = [1, 5, 14, 26, 31]
scores = [str(i) for i in scores]
scores_csv = ",".join(scores)
```

팰린드롬(Palindrome) 검사 프로그램

- 입력된 문장이 뒤집어도 동일한 형태의 대칭인지 판별하는 프로그램을 작성하시오.
(단, 문장은 영문자와 숫자로만 구성된다.)

문장을 입력하세요: level
결과: 팰린드롬입니다.

문장을 입력하세요: tomato
결과: 팰린드롬이 아닙니다.

문자 빈도수 세기 문제

문자열에 포함된 각 영문자의 출현 횟수를 세는 프로그램을 작성하시오.

```
A: 11 B: 2 C: 1 D: 1 E: 1 F: 1 G: 1 H: 1 I: 7 J: 0
K: 0 L: 2 M: 0 N: 15 O: 3 P: 1 Q: 0 R: 1 S: 0 T: 5
U: 0 V: 0 W: 3 X: 0 Y: 0 Z: 0 [: 0 \: 0 ]: 0 ^: 0
_: 0 `: 0 a: 184 b: 27 c: 83 d: 85 e: 307 f: 76 g: 53 h: 130
i: 205 j: 9 k: 20 l: 100 m: 63 n: 179 o: 217 p: 31 q: 4 r: 142
s: 154 t: 232 u: 70 v: 22 w: 44 x: 3 y: 41 z: 4
```

학습하기

2

세트의 이해

세트(set)

중복을 허용하지 않으며 순서를 유지하지 않는 변경 가능 컬렉션

- ✓ 수학적 집합 개념을 구현한 데이터 구조
- ✓ 원소 간의 고유성(uniqueness)을 보장
- ✓ 중복된 값을 자동으로 제거하여 데이터 정제에 유용

구문형식

```
set([원소 나열])
```

```
set((원소 나열))
```

```
{원소 나열}
```

세트 관련 연산자

- 할당 연산자 `=`, `in`, `not in` 연산자는 의미 동일
- 산술 연산자 `+`, `*` 는 사용 불가
 - ✓ 중복 불허 및 무순서 특징, `+` 대신 합집합(`|`) 사용
- 비교 연산자 `<`, `>`, `<=`, `>=`는 부분집합 여부로 의미 변경
- 동등 연산자 `==`, `!=` 는 순서 무시

```
scores = {87, 87, 98, 71, 98}
scores2 = {87, 98, 71}
scores == scores2
```

세트 관련 내장함수

- 중복이 허용되지 않음에 따라 다른 컬렉션 객체와 일부 실행 결과가 상이

연산자	의미
len(s)	중복이 제거된 후의 개수
min(s)	s에서 가장 작은 원소
max(s)	s에서 가장 큰 원소
sum(s)	모든 원소의 합(중복은 1회만)

```
scores = {87, 87, 98, 71, 98}
len(scores)
sum(scores)
```

세트 주요 메소드

메소드	의미
<code>issuperset(s: set): bool</code>	s가 자신의 상위집합 여부를 검사
<code>issubset(s: set): bool</code>	s가 자신의 부분집합 여부를 검사
<code>add(x: obj): None</code>	원소 x를 추가
<code>remove(x: obj): None</code>	원소 x를 제거
<code>intersection(s: set): set</code>	세트 s와 교집합 결과를 반환
<code>union(s: set): set</code>	세트 s와 합집합 결과를 반환
<code>difference(s: set): set</code>	세트 s와 차집합 결과를 반환

금치어 포함 확인 프로그램

■ 메일 제목에 금지어가 포함되어 있는지 검사하는 프로그램

제목을 입력하세요: Get it for FREE

결과: 스팸입니다.

제목을 입력하세요: 월요일 회의 관련사항

결과: 스팸이 아닙니다.

학습하기

3

딕셔너리의 이해

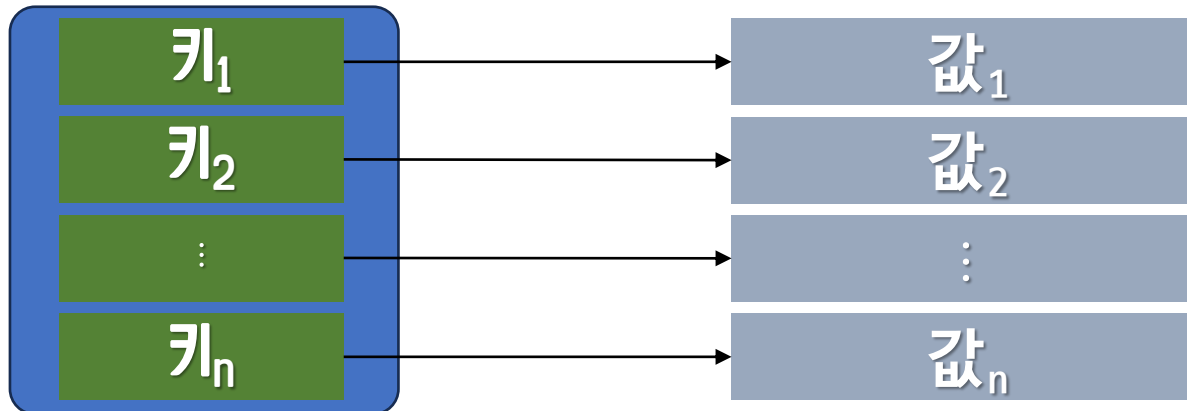
딕셔너리(dictionary)

key-value의 쌍으로 데이터를 저장하는 매핑(mapping) 자료형

- ✓ 키에 대응되는 값을 검색할 수 있도록 설계된 구조
- ✓ 키는 불변 객체만 허용되며, 값은 어떤 타입이든 저장

구문형식

$\{ \text{키}_1 : \text{값}_1, \text{키}_2 : \text{값}_2, \dots, \text{키}_n : \text{값}_n \}$



❖ 딕셔너리 관련 연산자

- ❑ `in, not in`는 키의 존재 여부 확인
- ❑ 산술 연산자 `+, *`,
비교 연산자 `<, >, <=, >=`는 사용 불가
- ❑ 접근 연산자 `[, ]`를 통해 값에 접근

```
d1 = {"a":10, "b":20, "c":30, "d":40}  
print(d1["a"])  
print(d1[ ])
```

- ❑ 동등 연산자 `==, !=`는 키-값 쌍의 일치 비교
✓ 순서는 무관

딕셔너리 주요 메소드

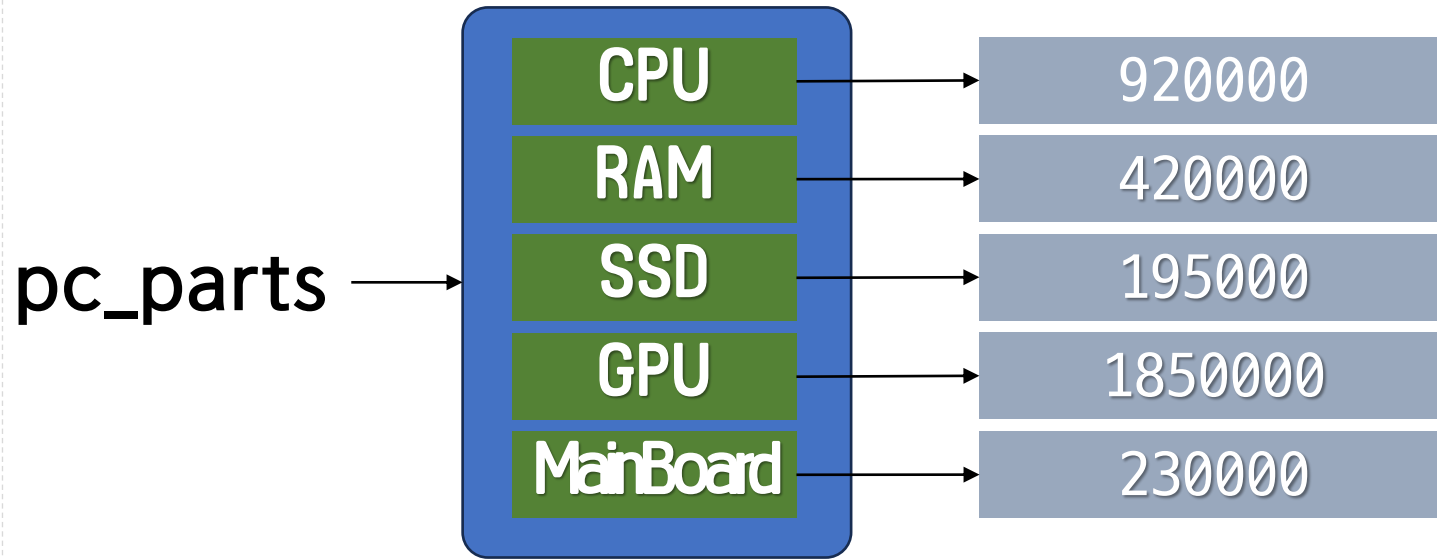
메소드	의미
keys(): tuple	포함된 모든 키를 반환
values(): tuple	포함된 모든 값을 반환
items(): tuple	(키, 값)의 튜플로 모든 항목을 반환
clear(): None	모든 항목을 삭제
get(key): value	키에 해당하는 값을 반환
pop(key): value	키에 해당하는 값을 반환하고 항목을 삭제
popitem(): tuple	무작위로 한 (키, 값)을 반환하고 선택된 항목을 삭제

딕셔너리 관련 내장 함수

연산자	의미
<code>len(s)</code>	키-값 쌍의 개수
<code>min(s.values())</code>	값 중 가장 작은 값
<code>min(s.keys())</code>	키 중 가장 작은 값
<code>max(s.values())</code>	값 중 가장 큰 값
<code>max(s.keys())</code>	키 중 가장 큰 값

```
basic_parts = {"CPU": 450000, "RAM": 120000}
len(basic_parts)
min(basic_parts.values())
```

딕셔너리의 생성 및 조작



```
pc_parts = { "CPU": 950000, "RAM": 420000,
             "SSD": 195000, "GPU": 1850000 }
pc_parts["MainBoard"] = 230000
pc_parts["CPU"] = 920000
```

모스 부호 변환 프로그램

■ 사용자 입력으로 받은 영문장을 모스 부호로 변환하는 프로그램

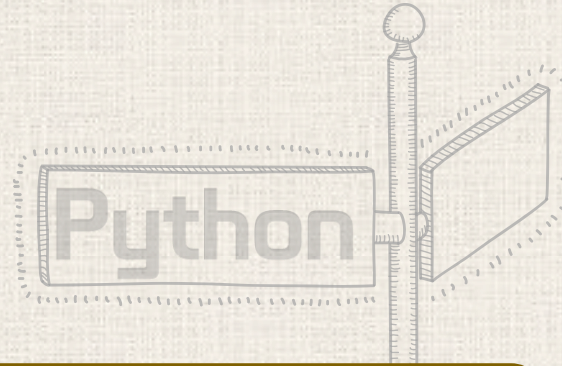
문장을 영어로 입력하세요: coming soon

-.-. --- -- .. -. --. ... --- --- -.

문장을 영어로 입력하세요: negative

- . . --. .- -- .

다음시간에는



10강. 튜플, 세트, 딕셔너리

▶ 11강. 다차원 구조

12강. 모듈